



รายงาน

PS04: Memory class and API in C programming

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	1
1 ข้อ 1: Static Storage Class	1
1.1 พื้นฐานที่ต้องเข้าใจก่อน: storage class คืออะไร	1
1.2 Step 1: โปรแกรมที่มี static คอมไพล์แบบปกติ	2
1.2.1 ผลการทดลอง	2
1.2.2 อธิบายผล	3
1.3 Step 2-3: ลบคำว่า static ออก	3
1.3.1 ผลการทดลอง	3
1.3.2 อธิบายผลและเปรียบเทียบกับ Step 1	3
1.4 Step 4: คอมไพล์ใหม่ด้วย -no-pie	4
1.4.1 ผลการทดลอง	4
1.4.2 ต่างจากเดิมหรือไม่ เพราะอะไร	5
1.5 หลักฐานยืนยันคำอธิบาย	5
1.6 อภิปรายผล	7

สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11), kernel 6.6.87.2-microsoft-standard-WSL2
สถาปัตยกรรม	x86-64 (64 บิต)
คอมไพเลอร์	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1) ตั้งค่ามาพร้อม <code>--enable-default-pie</code>
เครื่องมือตรวจสอบ	binutils (nm, readelf), file, util-linux (setarch)
ค่า ASLR	/proc/sys/kernel/randomize_va_space = 2 (สุ่มเต็มรูปแบบ)
Shell	GNU bash 5.2.21(1)-release
วันที่เก็บผล	6 กันยายน 2569

ผลการทดลองดิบ ทุกบรรทัดที่ยกมาในรายงานนี้ตัดมาจากไฟล์ `result/q1_verified.txt` ซึ่งสร้างซ้ำได้ด้วย `make result` (สคริปต์ `run_q1.sh` ในโฟลเดอร์ `src`) โจทย์กำหนดว่า ทุกขั้นตอนต้องรันบนเครื่องเดียวกัน สคริปต์นี้จึงคอมไพล์และรันทั้งสี่กรณี (มีหรือไม่มี `static` คุณกับ มีหรือไม่มี `-no-pie`) รวดเดียวจบในการเรียกครั้งเดียว ในรายงานแสดงเฉพาะบรรทัดที่จำเป็นต่อการอธิบาย ส่วนที่ตัดออกจะมีเครื่องหมาย ... หรือหมายเหตุ (lines omitted) กำกับไว้ และเพื่อให้ผลการรัน สามครั้งอยู่ในหน้าเดียวกันได้ ได้ย้ายบรรทัด `The address of y is ...` ขึ้นมาต่อท้ายบรรทัด `The value of y is ...` ของรอบเดียวกัน โดยไม่แก้ตัวเลขใด ๆ

หมายเหตุสองเรื่องเกี่ยวกับวิธีทดลอง (1) รายงานนี้คอมไพล์ด้วย `-Wall -Wextra` ตลอด จึงมี `warning return type of main is not int [-Wmain]` ติดมาทุกครั้ง เพราะโค้ดที่โจทย์ให้มา ประกาศ `void main()` ซึ่งไม่ตรงมาตรฐาน C (มาตรฐานกำหนดให้ `main` คืนค่า `int`) รายงานคงได้ไว้ตามโจทย์ทุกตัวอักษร เพราะเป็นเพียง warning ไม่ใช่ error และไม่กระทบผลการทดลองเรื่องที่อยู่ของตัวแปรเลย (2) การอธิบายว่า “ที่อยู่เปลี่ยนเพราะ ASLR” จะเชื่อถือได้ก็ต่อเมื่อ ปิด ASLR แล้วที่อยู่ต้องหยุดเปลี่ยน สคริปต์จึงรันโปรแกรมชุดเดิมซ้ำด้วย `setarch -R` (ปิดการสุ่ม address space เฉพาะ process ลูก) ไว้เป็นกรณีควบคุมในหัวข้อ 4 ของไฟล์ผล

1 ข้อ 1: Static Storage Class

1.1 พื้นฐานที่ต้องเข้าใจก่อน: storage class คืออะไร

คำว่า *storage class* ในภาษา C ไม่ได้บอกว่าตัวแปร “เป็นชนิดอะไร” แต่บอกว่าตัวแปรนั้น ถูกเก็บไว้ที่ไหนในหน่วยความจำ และมีชีวิตอยู่จนแค่ไหน คุณสมบัติที่ต้องแยกให้ออกมีสามเรื่อง

- ขอบเขตการมองเห็น (scope) คือบริเวณในโค้ดที่เรียกชื่อตัวแปรนี้ได้
- อายุ (lifetime) คือช่วงเวลาที่เนื้อที่ในหน่วยความจำของตัวแปรยังถูกจองไว้
- การเชื่อมโยง (linkage) คือไฟล์อื่นเห็นชื่อนี้ด้วยหรือไม่

หัวใจของข้อนี้คือ **scope กับ lifetime เป็นคนละเรื่องกัน** ตัวแปรหนึ่งตัว มีขอบเขตแคบมาก (เห็นได้เฉพาะในบล็อกเดียว) แต่มีอายุยืนตลอดโปรแกรมได้ และนั่นคือสิ่งที่คำว่า `static` ทำ

Table 1: เปรียบเทียบ storage class ของตัวแปรในภาษา C

ประกาศแบบ	ขอบเขต	อายุ	เก็บที่	ค่าเริ่มต้นถ้าไม่ได้
int y; ในบล็อก (คือ auto)	เฉพาะบล็อกนั้น	เกิดตอนเข้าบล็อก ตายตอนออกจากบล็อก	stack	ค่าขยะที่ค้างอยู่
static int y; ในบล็อก	เฉพาะบล็อกนั้น	ตลอดโปรแกรม	.data หรือ .bss	0 เสมอ
static int y; นอกฟังก์ชัน	ทั้งไฟล์	ตลอดโปรแกรม	.data หรือ .bss	0 เสมอ
int y; นอกฟังก์ชัน	ทั้งโปรแกรม	ตลอดโปรแกรม	.data หรือ .bss	0 เสมอ

อีกจุดที่ควรรู้คือคำว่า auto ในบรรทัด auto int x = 3; ของโจทย์ **ไม่ได้ทำอะไรเลย** เพราะตัวแปรที่ประกาศในบล็อกเป็น auto อยู่แล้วโดยปริยาย เขียนหรือไม่เขียนผลลัพธ์เหมือนกันทุกประการ (ในมาตรฐาน C23 คำนี้ถูกนำกลับมาใช้ในความหมายใหม่ คือให้คอมไพเลอร์เดาชนิดตัวแปรเอง แต่ gcc 13 ที่ใช้ในการทดลองนี้ตั้งต้นเป็น -std=gnu17 จึงยังเป็นความหมายเดิม)

1.2 Step 1: โปรแกรมที่มี static คอมไพล์แบบปกติ

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void main()
5 {
6     auto int x = 3;
7
8     while (x > 0)
9     {
10         static int y = 5;
11         y++;
12
13         printf("The value of y is %d\n", y);
14         printf("The address of y is %p\n\n", &y);
15
16         x--;
17     }
18 }

```

Code 1: src/q1_static.c คัดลอกจากโจทย์ทุกตัวอักษร

1.2.1 ผลการทดลอง

```

$ gcc -Wall -Wextra -o q1_static q1_static.c
q1_static.c:10:6: warning: return type of 'main' is not 'int' [-Wmain]
   10 | void main()
      |      ^~~~
(exit status = 0)

$ ./q1_static                                     # run 1
The value of y is 6      The address of y is 0x5c5469622010
The value of y is 7      The address of y is 0x5c5469622010
The value of y is 8      The address of y is 0x5c5469622010

$ ./q1_static                                     # run 2
The value of y is 6      The address of y is 0x60a5f69ee010
The value of y is 7      The address of y is 0x60a5f69ee010
The value of y is 8      The address of y is 0x60a5f69ee010

$ ./q1_static                                     # run 3
The value of y is 6      The address of y is 0x5faa56b46010
The value of y is 7      The address of y is 0x5faa56b46010
The value of y is 8      The address of y is 0x5faa56b46010

```

Code 2: คอมไพล์แบบปกติแล้วรัน 3 ครั้ง

1.2.2 อธิบายผล

ผลที่ได้มีสองส่วนที่ต้องอธิบายแยกกัน คือ *ค่า* กับ *ที่อยู่*

ค่าเป็น 6, 7, 8 ไม่ใช่ 6, 6, 6 เพราะบรรทัด `static int y = 5;` ไม่ใช่คำสั่งที่ทำงานตอนวนลูป แต่เป็นการบอกคอมไพเลอร์ให้จองเนื้อที่ไว้ในส่วน `.data` ของไฟล์โปรแกรม พร้อมเขียนเลข 5 ลงไปในไฟล์ตั้งแต่ ตอนคอมไพล์ เมื่อโปรแกรมถูกโหลดขึ้นหน่วยความจำ ค่า 5 ก็ติดมากับส่วนนั้นเลย การ “กำหนดค่าเริ่มต้น” จึงเกิดขึ้น **ครั้งเดียวก่อน** โปรแกรมเริ่มทำงาน ไม่ได้เกิดซ้ำๆ รอบที่ผ่านบรรทัดนั้น พอวนลูปรอบที่สอง โปรแกรมข้ามบรรทัดประกาศ ไปทำ `y++` ต่อจากค่าเดิมทันที ค่าจึงสะสมเป็น $5 \rightarrow 6 \rightarrow 7 \rightarrow 8$

ที่อยู่เท่ากันทั้ง 3 รอบภายในการรันเดียว เป็นผลโดยตรงจากข้อแรก คือมีตัวแปร `y` อยู่ *ตัวเดียว* ตลอดโปรแกรม ไม่ได้ถูกสร้างใหม่ทุกรอบ เมื่อมีตัวเดียวกีย่อมมีที่อยู่เดียว

แต่ที่อยู่เปลี่ยนทุกครั้งที่รันใหม่ ทั้งที่เป็นไฟล์โปรแกรมเดียวกัน สาเหตุคือ Ubuntu ตั้งค่า `gcc` ให้สร้าง **PIE (Position-Independent Executable)** เป็นค่าตั้งต้น (ยืนยันได้จาก `gcc -v` ที่มี `--enable-default-pie`) ไฟล์แบบนี้ไม่ได้ผูกที่อยู่จริงไว้ในตัวไฟล์ แต่เก็บเป็น *ระยะห่างจากจุดเริ่มของโปรแกรม* เอาไว้ เคอร์เนลจึงเลือก *ฐานที่* โหลด ใหม่แบบสุ่มได้ทุกครั้งทีรัน ซึ่งเป็นกลไก **ASLR (Address Space Layout Randomization)** ที่เปิดอยู่ในเครื่องนี้ (`randomize_va_space` มีค่า 2) ที่อยู่ที่พิมพ์ออกมาจึงเป็น “ฐานที่สุ่มได้ บวกกับ ระยะห่างคงที่” หลักฐานอยู่ในหัวข้อ 1.5

1.3 Step 2–3: ลบคำว่า `static` ออก

แก้ไขบรรทัดเดียวคือลบคำว่า `static` ออก (ไฟล์ `src/q1_auto.c`) ส่วนอื่นเหมือนเดิมทุกตัวอักษร

```
1 while (x > 0)
2 {
3     int y = 5;          /* the word "static" is gone */
4     y++;
5     ...
```

Code 3: `src/q1_auto.c` เฉพาะส่วนที่เปลี่ยน

1.3.1 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q1_auto q1_auto.c
$ ./q1_auto                                # run 1
The value of y is 6      The address of y is 0x7fff16657ab0
The value of y is 6      The address of y is 0x7fff16657ab0
The value of y is 6      The address of y is 0x7fff16657ab0

$ ./q1_auto                                # run 2
The value of y is 6      The address of y is 0x7ffde6d8b730
The value of y is 6      The address of y is 0x7ffde6d8b730
The value of y is 6      The address of y is 0x7ffde6d8b730

$ ./q1_auto                                # run 3
The value of y is 6      The address of y is 0x7ffd35c62ec0
The value of y is 6      The address of y is 0x7ffd35c62ec0
The value of y is 6      The address of y is 0x7ffd35c62ec0
```

Code 4: ลบ `static` ออกแล้วรัน 3 ครั้ง

1.3.2 อธิบายผลและเปรียบเทียบกับ Step 1

ค่ากลายเป็น 6, 6, 6 เพราะตอนนี้ `y` เป็นตัวแปรอัตโนมัติ อายุของมันผูกกับ *บล็อก* ไม่ใช่โปรแกรม ทุกครั้งที่วนซ้ำบล็อกใหม่ ตัวแปรจะถูกสร้างใหม่และคำสั่ง `= 5` ทำงานจริงทุกรอบ (ต่างจาก `static` ที่เป็นการเขียนค่าลงไฟล์ตั้งแต่ตอนคอมไพล์) พอบวกหนึ่งจึงได้ 6 เท่ากันทุกรอบ ค่าที่สะสมไว้รอบก่อนหายไปหมดตอนออกจากบล็อก

ที่อยู่ยังเท่ากันทั้ง 3 รอบ แต่เป็นคนละเหตุผลกับ Step 1 จุดนี้เป็นจุดที่เข้าใจผิดกันบ่อยที่สุด ที่อยู่เท่ากันไม่ได้แปลว่าเป็นตัวแปรตัวเดิม ความจริงคือคอมไพเลอร์จองสล็อตให้ y ไว้ใน *stack frame* ของ *main* ตั้งแต่ตอนเข้าฟังก์ชัน (เช่น “ถอยจากยอด frame ไป 20 ไบต์”) และ frame ของ *main* ไม่ได้ถูกทำลายระหว่างวนลูป การวนเข้าออกบล็อกจึงกลับมาใช้ **สล็อตเดิมซ้ำ** ทุกรอบ ที่อยู่จึงบังเอิญเท่ากัน แต่ **เนื้อหา** ในสล็อตนั้น ถูกเขียนทับด้วยเลข 5 ใหม่ทุกรอบ ซึ่งเห็นได้จากค่าที่ไม่สะสม ถ้าย้ายบล็อกนั้นไปอยู่ในฟังก์ชันที่เรียกตัวเองซ้อนกัน (recursive) ที่อยู่ก็จะต่างกันไปทุกครั้งทันที เพราะแต่ละครั้งได้ frame คนละอันบน stack

ที่อยู่ยังเปลี่ยนทุกครั้งที่รันใหม่ และอยู่คนละย่านกับ Step 1 สังเกตว่าเลขของ Step 1 ขึ้นต้นด้วย $0x5...$ แต่ของ Step 2 ขึ้นต้นด้วย $0x7ff...$ ซึ่งไม่ใช่ความบังเอิญ เลข $0x7ff...$ คือย่านบนสุดของ address space ที่ระบบปฏิบัติการวาง *stack* ไว้ ส่วน $0x5...$ คือย่านที่ เคอร์เนลใช้โหลดตัวโปรแกรมแบบ PIE หลักฐานว่าเลขนี้เป็น *stack* จริง อยู่ในหัวข้อ 1.5 ส่วนสาเหตุที่เปลี่ยนทุกครั้งก็คือ ASLR เช่นกัน เพียงแต่เป็น **การสุ่มตำแหน่ง stack** ซึ่งเป็นคนละกลไกกับการสุ่มฐานของ PIE ประเด็นนี้จะสำคัญมากใน Step 4

Table 2: สรุปความต่างระหว่างมีและไม่มี static

ประเด็น	static int y = 5;	int y = 5;
ค่าที่พิมพ์ออกมา	6, 7, 8	6, 6, 6
กำหนดค่าเริ่มต้นเมื่อไร	ครั้งเดียว ตอน คอมไพล์ และ โหลดโปรแกรม	ทุกครั้งที่วนเข้าบล็อก
จำนวนตัวแปรจริง ๆ	1 ตัวตลอดโปรแกรม	สร้างใหม่ทุกรอบ แต่ใช้ที่เดิมซ้ำ
อยู่ ส่วนไหน ของ หน่วยความจำ	.data (ย่าน $0x5...$ เมื่อเป็น PIE)	stack (ย่าน $0x7ff...$)
อายุ	ตลอดโปรแกรม	เฉพาะช่วงที่อยู่ในบล็อก
ขอบเขตการมองเห็น	เฉพาะในบล็อก (เหมือนกันทั้งคู่)	เฉพาะในบล็อก (เหมือนกันทั้งคู่)
มีชื่อในตาราง symbol ใหม่	มี ชื่อ $y.0$	ไม่มี เป็นแค่ระยะทางใน frame

1.4 Step 4: คอมไพล์ใหม่ด้วย -no-pie

ทำซ้ำ Step 1–3 ทั้งหมด โดยเปลี่ยนเฉพาะคำสั่งคอมไพล์เป็น `gcc -no-pie -o <output> <source>` ใช้ซอร์สโค้ดไฟล์เดิมทั้งสองไฟล์ ไม่แก้อะไรเลยแม้แต่ตัวอักษรเดียว

1.4.1 ผลการทดลอง

```
$ gcc -no-pie -Wall -Wextra -o q1_static_nopie q1_static.c
$ ./q1_static_nopie # run 1
The value of y is 6      The address of y is 0x404018
The value of y is 7      The address of y is 0x404018
The value of y is 8      The address of y is 0x404018

$ ./q1_static_nopie # run 2
The value of y is 6      The address of y is 0x404018
The value of y is 7      The address of y is 0x404018
The value of y is 8      The address of y is 0x404018

$ ./q1_static_nopie # run 3
The value of y is 6      The address of y is 0x404018
The value of y is 7      The address of y is 0x404018
The value of y is 8      The address of y is 0x404018
```

Code 5: -no-pie กับโปรแกรมที่มี static

```
$ gcc -no-pie -Wall -Wextra -o q1_auto_nopie q1_auto.c
$ ./q1_auto_nopie # run 1
The value of y is 6      The address of y is 0x7ffe931c00e0
```

```

The value of y is 6      The address of y is 0x7ffe931c00e0
The value of y is 6      The address of y is 0x7ffe931c00e0

$ ./q1_auto_nopie          # run 2
The value of y is 6      The address of y is 0x7fff0bc434d0
The value of y is 6      The address of y is 0x7fff0bc434d0
The value of y is 6      The address of y is 0x7fff0bc434d0

$ ./q1_auto_nopie          # run 3
The value of y is 6      The address of y is 0x7ffdbc93c020
The value of y is 6      The address of y is 0x7ffdbc93c020
The value of y is 6      The address of y is 0x7ffdbc93c020

```

Code 6: -no-pie กับโปรแกรมที่ไม่มี static

1.4.2 ต่างจากเดิมหรือไม่ เพราะอะไร

คำตอบคือ **ต่างเฉพาะครั้งเดียว** และครั้งที่ต่างกับครั้งที่ไม่ว่ากันเอง ที่ทำให้เห็นว่า -no-pie ทำงานกับหน่วยความจำส่วนไหนกันแน่

Table 3: สรุปผลทั้งสี่กรณี: ที่อยู่ของ y คงที่ข้ามการรันหรือไม่

กรณี	คอมไพล์ปกติ (PIE)	คอมไพล์ -no-pie	สรุป
มี static	0x5c54..., 0x60a5..., 0x5faa... เปลี่ยนทุกครั้ง	0x404018 เท่ากันทั้งสามครั้ง	ต่างกัน -no-pie ตรงที่อยู่ได้
ไม่มี static	0x7fff..., 0x7ffd..., 0x7ffd... เปลี่ยนทุกครั้ง	0x7ffe..., 0x7fff..., 0x7ffd... ยังเปลี่ยนทุกครั้ง	ไม่ต่าง -no-pie ไม่ช่วยอะไร
ค่าที่พิมพ์	6,7,8 และ 6,6,6	6,7,8 และ 6,6,6	ไม่ต่าง เหมือนเดิมทุกกรณี

(1) ค่าของ y ไม่เปลี่ยนแปลงไม่ว่าจะคอมไพล์แบบไหน เพราะ -no-pie เป็นเรื่องของ การจัดวางในหน่วยความจำ ล้วน ๆ ไม่ได้เปลี่ยน ความหมายของภาษา C แม้แต่นิดเดียว static ก็ยังคงเป็น static ตัวแปรอัตโนมัติก็ยังคงเกิดใหม่ทุกครั้งเหมือนเดิม

(2) กรณี static ต่างชัดเจน ที่อยู่กลายเป็น 0x404018 เท่ากันเป๊ะทั้งสามครั้ง เพราะ -no-pie สั่งให้ linker สร้างไฟล์ชนิด EXEC ที่ ผู้ที่อยู่จริงไว้ตายตัวตั้งแต่ตอน link เคอร์เนลไม่มีทางเลือก ต้องโหลดโปรแกรมลงที่อยู่นั้นเป๊ะ ๆ เสมอ ตัวแปร y ที่อยู่ในส่วน .data จึงมีที่อยู่เดิมทุกครั้ง ตรวจสอบได้ด้วย nm ว่าเลข 0x404018 ที่โปรแกรมพิมพ์ออกมา คือเลขเดียวกับที่อยู่ในตาราง symbol ของไฟล์ ไม่ได้ถูกกำหนดขึ้นตอนรันเลย

(3) กรณีไม่มี static ไม่ต่างเลย ที่อยู่ยังเป็น 0x7fff... และยังสุมทุกครั้งเหมือนเดิม เพราะ y ตัวนี้อยู่บน stack ซึ่ง ไม่ได้อยู่ในไฟล์โปรแกรม เลย stack เป็นสิ่งที่เคอร์เนลสร้างให้ ตอนสร้าง process และตำแหน่งของมันถูกสุมโดยกลไก ASLR ของเคอร์เนล ซึ่งเป็นคนละกลไกกับการสุมฐานของ PIE โดยสิ้นเชิง -no-pie จึงปิดได้ เฉพาะการสุมฐานของตัวโปรแกรม แต่ปิดการสุม stack ไม่ได้ ถ้าอยากปิดการสุม stack ด้วยต้องสั่งที่ระบบปฏิบัติการ เช่น setarch -R หรือแก้ค่า /proc/sys/kernel/randomize_va_space ซึ่งได้ทดลองไว้แล้วในหัวข้อถัดไป

ข้อสรุปของ Step 4 -no-pie ไม่ได้ “ปิด ASLR” อย่างที่มักเข้าใจกัน มันแค่ทำให้ ตัวโปรแกรมและข้อมูลของโปรแกรม (.text, .data, .bss) มีที่อยู่ตายตัว ส่วน stack, heap และไลบรารีที่โหลดตอนรัน ยังถูกสุมอยู่เหมือนเดิม

1.5 หลักฐานยืนยันคำอธิบาย

คำอธิบายทั้งหมดข้างต้นจะเป็นแค่การอ้างลอย ๆ ถ้าไม่มีการตรวจสอบจากอีกทาง สคริปต์เก็บผลจึงมีหัวข้อ 4 ที่ได้ตรวจสอบเรื่อง คือ ตัวไฟล์เป็นชนิดอะไร ตัวแปรอยู่ตรงไหนในไฟล์ และถ้าปิด ASLR แล้วผลจะเปลี่ยนไปอย่างไร

หลักฐานที่ 1: ไฟล์ที่ได้เป็นคนละชนิดกันจริง

```
$ file q1_static q1_auto q1_static_nopie q1_auto_nopie
q1_static:      ELF 64-bit LSB pie executable, x86-64, dynamically linked, ...
q1_auto:       ELF 64-bit LSB pie executable, x86-64, dynamically linked, ...
q1_static_nopie: ELF 64-bit LSB executable,      x86-64, dynamically linked, ...
q1_auto_nopie:  ELF 64-bit LSB executable,      x86-64, dynamically linked, ...

$ readelf -h q1_static | grep -E 'Type|Entry'
Type:          DYN (Position-Independent Executable file)
Entry point address: 0x1060      <- an offset, not a real address

$ readelf -h q1_static_nopie | grep -E 'Type|Entry'
Type:          EXEC (Executable file)
Entry point address: 0x401050    <- a real, fixed address
```

Code 7: ชนิดของไฟล์ ELF ทั้งสี่ (ตัด BuildID ออก)

หลักฐานที่ 2: ตัวแปร static มีชื่อและที่อยู่ภายในไฟล์ ส่วนตัวแปรอัตโนมัติไม่มี

```
$ nm q1_static_nopie | grep -i 'y'          # 'd' = a local symbol in .data
0000000000404018 d y.0                      <- exactly what the program printed

$ nm q1_static | grep -i 'y'               # PIE: the same symbol, but as an offset
0000000000004010 d y.0

$ nm q1_auto_nopie | grep -i 'y'           # nothing at all
(exit status = 1 -> 1 means grep found no symbol named y)

$ readelf -S q1_static_nopie | grep -A1 ' .data '
[25] .data          PROGBITS          0000000000004008 00003008
      0000000000000014 0000000000000000 WA          0      0      8
```

Code 8: ตาราง symbol และส่วน .data

บรรทัดแรกคือหลักฐานที่ตรงที่สุดของทั้งข้อ เลข 0x404018 ที่โปรแกรมพิมพ์ ออกมาตอนรัน คือเลขเดียวกับที่ nm อ่านได้จาก ตัวไฟล์ โดยไม่ต้องรันเลย และมันตกอยู่ในช่วงของส่วน .data ที่เริ่มที่ 0x404008 พอดี ส่วนชื่อ y.0 ที่มีเลขต่อท้าย เป็นเพราะคอมไพเลอร์ต้องตั้งชื่อไม่ให้ชนกับ static ตัวอื่นที่อาจชื่อ y เหมือนกันในฟังก์ชันอื่น ในทางกลับกัน q1_auto_nopie ไม่มี symbol ชื่อ y เลย เพราะตัวแปรอัตโนมัติไม่ใช่ “ของ” ที่อยู่ในไฟล์ แต่เป็นเพียงระยะห่างในโค้ดที่นำไปคำนวณกับ stack pointer ตอนรัน

หลักฐานที่ 3: ปิด ASLR แล้วที่อยู่หยุดสุ่มจริง (กรณีควบคุม)

```
$ setarch -R ./q1_static | grep address    # run 1, 2, 3
The address of y is 0x555555558010
The address of y is 0x555555558010
The address of y is 0x555555558010

$ setarch -R ./q1_auto | grep address      # run 1, 2, 3
The address of y is 0x7fffffffe290
The address of y is 0x7fffffffe290
The address of y is 0x7fffffffe290
```

Code 9: รันไฟล์ PIE ชุดเดิมด้วย setarch -R ซึ่งปิดการสุ่ม address space

ไฟล์เดียวกันกับที่เพิ่งให้ผลสุ่มไปเมื่อครู่ พอปิด ASLR กลับให้เลขเดิมทุกครั้ง นี่คือหลักฐานว่าตัวการที่ทำให้ที่อยู่เปลี่ยนคือ ASLR ไม่ใช่ตัวภาษา C และไม่ใช่ static ที่สำคัญคือ q1_auto ที่คอมไพล์แบบ PIE ก็หยุดสุ่มด้วย ทั้งที่ -no-pie ทำให้มันหยุดสุ่มไม่ได้ ยืนยันข้อสรุปของ Step 4 ว่าที่อยู่ของ stack ถูกคุมด้วย ASLR ของเคอร์เนล ไม่ใช่ด้วยชนิดของไฟล์ ELF

หลักฐานที่ 4: เลข 0x7ff... เป็นที่อยู่บน stack จริง

```
$ setarch -R ./q1_auto | grep address | head -1
The address of y is 0x7fffffffe290      <- inside the range below

$ setarch -R bash -c 'grep "[stack\]" /proc/self/maps'
7ffffffde000-7fffffffe000 rw-p 00000000 00:00 0          [stack]

$ setarch -R ./q1_static | grep address | head -1
The address of y is 0x555555558010      <- NOT inside that range
```

Code 10: เทียบกับแผนที่หน่วยความจำจริง (ปิด ASLR ทั้งคู่เพื่อให้เทียบตัวเลขกันได้)

ที่อยู่ 0x7fffffff290 ที่โปรแกรมพิมพ์ออกมา ตกอยู่ในช่วง 0x7fffffffde000 ถึง 0x7fffffff000 ที่เคอร์เนลระบุว่า เป็น [stack] พอดี ในขณะที่ที่อยู่ของตัวแปร static อยู่คนละย่านโดยสิ้นเชิง

หลักฐานที่ 5: ที่อยู่ของ PIE เท่ากับ ฐานที่สุ่มได้ บวก ระยะห่างคงที่

```
$ ./q1_static | head -2          # run 1, 2, 3 -> base = printed address - 0x4010
printed &y = 0x59dfb0606010      load base = 0x59dfb0602000
printed &y = 0x6234ef9d2010      load base = 0x6234ef9ce000
printed &y = 0x64fd3cae1010      load base = 0x64fd3cadd000

$ ./q1_static_nopie | head -2   # run 1, 2, 3 -> base = printed address - 0x404018
printed &y = 0x404018           load base = 0x0
printed &y = 0x404018           load base = 0x0
printed &y = 0x404018           load base = 0x0
```

Code 11: แยกส่วนที่สุ่มออกจากส่วนที่คงที่ (ระยะห่างของ y คือ 0x4010 จาก nm)

ข้อสังเกตสองอย่าง หนึ่งคือฐานที่คำนวณได้ลงท้ายด้วย 000 ทุกครั้ง เพราะเคอร์เนลสุ่มได้ที่ละหน้า (page ละ 4096 ไบต์) ไม่ได้สุ่มที่ละไบต์ สองคือไฟล์แบบ -no-pie มีฐานเป็นศูนย์ นั่นคือที่อยู่ที่พิมพ์ออกมา เท่ากับที่อยู่ที่ระบุไว้ในไฟล์ เป๊ะ ๆ ซึ่งเป็นนิยามของคำว่า “ไม่ position-independent”

1.6 อภิปรายผล

- **static เปลี่ยน “อายุ” ไม่ได้เปลี่ยน “ขอบเขต”**: เป็นบทเรียนหลักของข้อนี้ ตัวแปร y ในทั้งสองเวอร์ชันมีขอบเขตเท่ากันเป๊ะ คือเรียกใช้ได้เฉพาะในบล็อกของ while เท่านั้น โค้ดนอกบล็อกยังคอมไพล์ไม่ผ่านเหมือนกัน สิ่ง static เปลี่ยน คือย้ายตัวแปรจาก stack ไปอยู่ .data ทำให้อายุยืนตลอดโปรแกรม ผลคือได้ตัวแปรที่ “จำค่าได้ระหว่างการเรียกแต่ละครั้ง แต่ไม่มีใครนอกบล็อก ไปยุ่งกับมันได้” ซึ่งเป็นเหตุผลที่นิยมใช้ทำตัวนับหรือ cache ในฟังก์ชัน
- **การกำหนดค่าเริ่มต้นของ static เกิดตอนคอมไพล์ ไม่ใช่ตอนรัน**: เป็นสาเหตุที่ค่าออกมาเป็น 6, 7, 8 และเป็นเหตุผลที่มาตรฐาน C บังคับว่า ค่าเริ่มต้นของตัวแปร static ต้องเป็นค่าคงที่ที่รู้ตั้งแต่ตอนคอมไพล์ (เขียน static int y = rand(); ไม่ได้ แต่ int y = rand(); ได้) ผลข้างเคียงอีกอย่างคือถ้าไม่ใส่ค่าเริ่มต้น ตัวแปร static จะได้ค่า 0 เสมอโดยไม่ต้องเสียเนื้อที่ในไฟล์ เพราะไปอยู่ในส่วน .bss ที่เคอร์เนลล้างเป็นศูนย์ให้ตอนโหลด ต่างจากตัวแปรอัตโนมัติ ที่ถ้าไม่ใส่ค่าเริ่มต้นจะได้ค่าขยะที่ค้างอยู่ใน stack
- **ที่อยู่ที่พิมพ์ออกมาเป็น virtual address ไม่ใช่ที่อยู่จริงบน RAM**: เห็นได้ชัดจากกรณี -no-pie ที่ทุก process ได้เลข 0x404018 เหมือนกันหมด ถ้ารันพร้อมกันสิบตัวก็ยังเป็นเลขนี้ทั้งสิบตัวและไม่ชนกัน เพราะแต่ละ process มี address space ของตัวเอง โดยมีหน่วย MMU กับตาราง page ของเคอร์เนลแปลงเลขนี้ไปเป็นที่อยู่จริงคนละที่กัน นี่คือภาพรูปธรรมของเรื่อง virtual memory ที่เรียนในทฤษฎี
- **เหตุผลที่ Linux รุ่นใหม่ทำ PIE เป็นค่าตั้งต้น คือความปลอดภัย**: การโจมตีแบบ buffer overflow ยุคใหม่ (เช่น return-to-libc หรือ ROP) ต้องรู้ที่อยู่ของโค้ดหรือข้อมูลที่จะกระโดดไปให้แม่นยำ การสุ่มฐานทุกครั้ง ที่รันทำให้ผู้โจมตีต้องเดา ซึ่งมักทำให้โปรแกรมพังไปเสียก่อน แลกมาด้วยราคาเล็กน้อยคือโค้ดต้องอ้างอิงแบบ relative จึงเป็นเหตุผลที่ -no-pie ยังมีอยู่ คือใช้ตอน debug ที่อยากให้อยู่คงที่เพื่อเทียบผลได้ หรือใช้กับโค้ดเก่า ที่เขียนแบบผูกที่อยู่ตายตัว ไม่ใช่ตัวเลือกที่ควรใช้กับโปรแกรมจริง
- **ข้อควรระวังของ static ในโปรแกรมจริง**: เมื่อฟังก์ชันมี static อยู่ข้างใน ฟังก์ชันนั้นจะ “จำอดีต” ได้ ผลคือเรียกด้วย อาร์กิวเมนต์เดิมสองครั้งอาจได้ผลไม่เหมือนกัน ทำให้ทดสอบยากขึ้นมาก และที่ร้ายกว่านั้นคือถ้ามีหลาย thread เรียกฟังก์ชันเดียวกันพร้อมกัน ทุก thread จะใช้ตัวแปรตัวเดียวกัน เกิด race condition ทันที (ฟังก์ชันเก่าในไลบรารีมาตรฐานอย่าง strtok หรือ ctime มีปัญหานี้ จึงมีรุ่นลงท้ายด้วย _r ที่ให้ผู้เรียกสงที่เก็บสถานะมาเองแทน)
- **void main() ไม่ถูกต้องตามมาตรฐาน**: มาตรฐาน C กำหนดให้ main คืนค่า int เพื่อส่ง exit status ให้ระบบ

ปฏิบัติการ รายงานนี้คงได้ทำตามโจทย์และแนบ warning มาแสดงตามจริง แต่ในการใช้งานจริง ควรเขียนเป็น `int main(void)` และ `return 0;` เสมอ โดยเฉพาะเมื่อโปรแกรมจะถูกเรียกจากสคริปต์ที่ตรวจสอบ exit status